Arithmetic Operations on DataFrame

Pandas DataFrame is a two-dimensional, labeled data structure that allows for efficient data manipulation and analysis. One of the primary features of Pandas is its ability to perform vectorized arithmetic operations on DataFrames. This means you can apply mathematical operations without using loop through elements manually.

Applying arithmetic operations in Pandas allows you to manipulate data quickly and efficiently, whether you're working with a single DataFrame or performing operations between multiple DataFrames.

Arithmetic Operations on DataFrame with Scalar Value

You can perform arithmetic operations on a DataFrame with scalar values directly. These operations are applied element-wise, meaning that every value in the DataFrame is affected by the arithmetic operation.

List of commonly used arithmetic operators on Pandas DataFrame −

Operation	Example	Description
Addition	df + 2	Adds 2 to each element of the DataFrame
Subtraction	df – 2	Subtracts 2 from each element
Multiplication	df * 2	Multiplies each element by 2

Division	df / 2	Divides each element by 2
Exponentiation	df ** 2	Raises each element to the power of 2
Modulus	df % 2	Finds the remainder when divided by 2
Floor Division	df // 2	Divides and floors the quotient

Example

Applying all the arithmetical operators on a Pandas DataFrame with a scalar value.

```
import pandas as pd

data = {'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]}

df = pd.DataFrame(data)

print(df)

# Perform arithmetic operations

print("\nAddition: \n ", df + 2)

print("\nSubtraction: \n ", df - 2)

print("\nMultiplication: \n ", df * 2)

print("\nDivision: \n ", df / 2)

print("\nExponentiation: \n ", df ** 2)

print("\nModulus: \n ", df % 2)

print("\nFloor Division: \n", df // 2)
```

Output

	A	B
0	1	5
1	2	6
2	3	7
3	4	8

Addition:

	A	B
0	3	7
1	4	8
2	5	9
3	6	10

Subtraction:

	A	B
0	-1	3
1	0	4
2	1	5
3	2	6

Multiplication:

	A	B
0	2	10
1	4	12
2	6	14
3	8	16

Division:

	A	B
0	0.5	2.5
1	1.0	3.0
2	1.5	3.5
3	2.0	4.0

Exponentiation:

	A	B
0	1	25
1	4	36
2	9	49
3	16	64

Modulus:

	A	B
0	1	1
1	0	0
2	1	1
3	0	0

Floor Division:

	A	B
0	0	2
1	1	3
2	1	3
3	2	4

Arithmetic Operations Between Two DataFrames

Pandas allows you to apply arithmetic operators between two DataFrames efficiently. These operations are applied element-wise, meaning corresponding elements in both DataFrames are used in calculations.

When performing arithmetic operations on two DataFrames, Pandas aligns them based on their index and column labels. If a particular index or column is missing in either DataFrame, the result for those entries will be NaN, indicating missing values.

Example

This example demonstrates applying the arithmetic operations on two DataFrame. These operations include addition, subtraction, multiplication, and division of two DataFrame.

```
import pandas as pd

df1 = pd.DataFrame({'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]})

df2 = pd.DataFrame({'A': [10, 20, 30], 'B': [50, 60, 70]}, index=[1, 2, 3])

print("DataFrame 1:\n", df1)

print("\nDataFrame 2:\n", df2)

# Perform arithmetic operations

print("\nAddition of Two DataFrames:\n", df1 + df2)

print("\nSubtraction of Two DataFrames:\n", df1 - df2)

print("\nMultiplication of Two DataFrames:\n", df1 * df2)

print("\nDivision of Two DataFrames:\n", df1 / df2)
```

Following is the output of the above code −

```
DataFrame 1:
```

	A	B
0	1	5
1	2	6
2	3	7
3	4	8

DataFrame 2:

	A	B
1	10	50
2	20	60
3	30	70

Addition of Two DataFrames:

	A	B
0	NaN	NaN
1	12.0	56.0
2	23.0	67.0
3	34.0	78.0

Subtraction of Two DataFrames:

	A	B
0	NaN	NaN

1 -8.0 -44.0

2 -17.0 -53.0

3 -26.0 -62.0

Multiplication of Two DataFrames:

	A	B
0	NaN	NaN
1	22.0	300.0
2	60.0	420.0
3	120.0	560.0

Division of Two DataFrames:

	A	B
0	NaN	NaN
1	0.200000	0.120000
2	0.150000	0.116667
3	0.133333	0.114286

DatFrame Concatenation

Concatenation in Pandas refers to the process of joining two or more Pandas objects (like DataFrames or Series) along a specified axis. This operation is very useful when you need to merge data from different sources or datasets.

The primary tool for this operation is pd.concat() function, which can useful for Series, DataFrame objects, whether you're combining rows or columns. Concatenation in Pandas involves combining multiple DataFrame or Series objects either row-wise or column-wise.

pd.concat() Function

The pandas.concat() function is the primary method used for concatenation in Pandas. It allows you to concatenate pandas objects along a particular axis with various options for handling indexes.

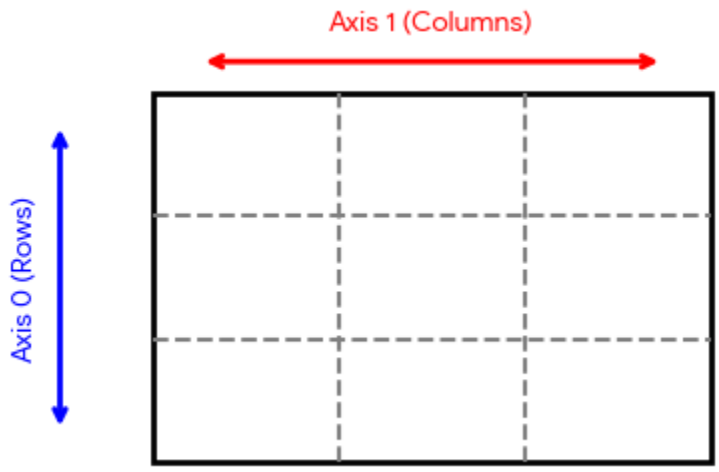
Syntax

pandas.concat(objs, *, axis=0, join='outer', ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, sort=False, copy=None)

Where,

objs: This is a sequence or mapping of Series, DataFrame, or Panel objects.

axis: {0, 1, ...}, default 0. This is the axis to concatenate along.



join: {"inner", "outer"}, default "outer". How to handle indexes on other axis(es). Outer for union and inner for intersection.

ignore_index: boolean, default False. If True, do not use the index values on the concatenation axis. The resulting axis will be labeled 0, ..., n - 1.

keys: Used to create a hierarchical index along the concatenation axis.

levels: Specific levels to use for the MultiIndex in the result.

names: Names for the levels in the resulting hierarchical index.

verify_integrity: If True, checks for duplicate entries in the new axis and raises an error if duplicates are found.

sort: When combining DataFrames with unaligned columns, this parameter ensures the columns are sorted.

copy: default None. If False, do not copy data unnecessarily.

The concat() function does all of the heavy lifting of performing concatenation operations along an axis.

Example

```
import pandas as pd

one = pd.DataFrame({
   'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
   'subject':['sub1','sub2','sub4','sub6','sub5'],
   'Marks':[98,90,87,69,78]},
   index=[1,2,3,4,5])


two = pd.DataFrame({
   'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
   'subject':['sub2','sub4','sub3','sub6','sub5'],
   'Marks':[89,80,79,97,88]},
   index=[1,2,3,4,5])


# Concatenating DataFrames
result = pd.concat([one, two])
print(result)
```

Output

	Name	subject_id	Marks_scored
1	A1	sub1	98
2	A2	sub2	90
3	A3	sub4	87
4	A4	sub6	69
5	A5	sub5	78
1	B1	sub2	89
2	B2	sub4	80
3	B3	sub3	79
4	B4	sub6	97
5	B5	sub5	88

Example: Concatenating with Keys

If you want to distinguish between the concatenated DataFrames, you can use the keys parameter to associate specific keys with each part of the DataFrame.

```
import pandas as pd

one = pd.DataFrame({

   'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
```

```
   'subject_id':['sub1','sub2','sub4','sub6','sub5'],
   'Marks_scored':[98,90,87,69,78]},
   index=[1,2,3,4,5])

two = pd.DataFrame({
   'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
   'subject_id':['sub2','sub4','sub3','sub6','sub5'],
   'Marks_scored':[89,80,79,97,88]},
   index=[1,2,3,4,5])
print(pd.concat([one,two],keys=['x','y']))
```

Output

```
      Name subject_id  Marks_scored
x 1    A1       sub1           98
  2    A2       sub2           90
  3   A3       sub4           87
  4   A4       sub6           69
  5  A5       sub5           78
y 1   B1       sub2           89
```

2	B2	sub4	80
3	B3	sub3	79
4	B4	sub6	97
5	B5	sub5	88

Here, the x and y keys create a hierarchical index, allowing easy identification of which original DataFrame each row came from.

Example: Ignoring Indexes During Concatenation

```
import pandas as pd
one = pd.DataFrame({
   'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
   'subject_id':['sub1','sub2','sub4','sub6','sub5'],
   'Marks_scored':[98,90,87,69,78]},
   index=[1,2,3,4,5])

two = pd.DataFrame({
   'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
   'subject_id':['sub2','sub4','sub3','sub6','sub5'],
   'Marks_scored':[89,80,79,97,88]},
```

```
   index=[1,2,3,4,5])

print(pd.concat([one,two], ignore_index=True))
```

Output

```
     Name subject_id  Marks_scored

0    A1       sub1           98

1     A2       sub2           90

2   A3       sub4           87

3   A4       sub6           69

4  A5       sub5           78

5   B1       sub2           89

6   B2       sub4           80

7    B3       sub3           79

8   B4       sub6           97

9   B5       sub5           88
```

Observe, the index changes completely and the Keys are also overridden.

Example: Concatenating Along Columns

Instead of concatenating along rows, you can concatenate along columns by setting the axis parameter to 1.

```
import pandas as pd
one = pd.DataFrame({
   'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
   'subject_id':['sub1','sub2','sub4','sub6','sub5'],
   'Marks_scored':[98,90,87,69,78]},
   index=[1,2,3,4,5])

two = pd.DataFrame({
   'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
   'subject_id':['sub2','sub4','sub3','sub6','sub5'],
   'Marks_scored':[89,80,79,97,88]},
   index=[1,2,3,4,5])
print(pd.concat([one,two],axis=1))
```

Output

```
   Name subject_id  Marks_scored   Name subject_id  Marks_scored
```

1	A1	sub1	98	B1	sub2	89
2	A2	sub2	90	B2	sub4	80
3	A3	sub4	87	B3	sub3	79
4	A4	sub6	69	B4	sub6	97
5	A5	sub5	78	B5	sub5	88

Python Pandas - Merging/Joining

Pandas provides high-performance, in-memory join operations similar to those in SQL databases. These operations allow you to merge multiple DataFrame objects based on common keys or indexes efficiently.

The merge() Method in Pandas

The DataFrame.merge() method in Pandas enables merging of DataFrame or named Series objects using database-style joins. A named Series is treated as a DataFrame with a single named column. Joins can be performed on columns or indexes.

If merging on columns, DataFrame indexes are ignored. If merging on indexes or indexes with columns, then the index will remains the same. However, in cross merges (how='cross'), you cannot specify column names for merging.

Syntax

DataFrame.merge(right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False)

The key parameters are −

right: A DataFrame or a named Series to merge with.

on: Columns (names) to join on. Must be found in both the DataFrame objects.

left_on: Columns from the left DataFrame to use as keys. Can either be column names or arrays with length equal to the length of the DataFrame.

right_on: Columns from the right DataFrame to use as keys. Can either be column names or arrays with length equal to the length of the DataFrame.

left_index: If True, use the index (row labels) from the left DataFrame as its join key(s). In case of a DataFrame with a MultiIndex (hierarchical), the number of levels must match the number of join keys from the right DataFrame.

right_index: Same usage as left_index for the right DataFrame.

how: Determines type of join operation, available options are 'left', 'right', 'outer', 'inner', and 'cross'. Defaults is 'inner'.

sort: Sort the result DataFrame by the join keys in lexicographical order. Defaults to True, setting to False will improve the performance substantially in many cases.

Example

```
import pandas as pd

left = pd.DataFrame({

'id': [1, 2, 3, 4, 5],

'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],

'subject': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']

})

right = pd.DataFrame({

'id': [1, 2, 3, 4, 5],

'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],

'subject_id': ['sub2', 'sub4', 'sub3', 'sub6', 'sub5']

})


print(left)

print(right)
```

Output

```
    id   Name subject_id

0   1    A1        sub1
```

1	2	A2	sub2
2	3	A3	sub4
3	4	A4	sub6
4	5	A5	sub5

id	Name	subject_id	
0	1	B1	sub2
1	2	B2	sub4
2	3	B3	sub3
3	4	B4	sub6
4	5	B5	sub5

Merge Two DataFrames on a Key

You can merge two DataFrames using a common key column by specifying the column name in the on parameter of the merge() method.

Example

```
import pandas as pd

left = pd.DataFrame({

'id': [1, 2, 3, 4, 5],
```

```
'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
'subject_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']
})
right = pd.DataFrame({
'id': [1, 2, 3, 4, 5],
'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
'subject_id': ['sub2', 'sub4', 'sub3', 'sub6', 'sub5']
})

# Merging DataFrames on a key 'id'
result = left.merge(right, on='id')
print(result)
```

Output

id	Name_x	subject_id_x	Name_y	subject_id_y
0	1 A1	sub1	B1	sub2
1	2 A2	sub2	B2	sub4
2	3 A3	sub4	B3	sub3
3	4 A4	sub6	B4	sub6

4 5 A5 sub5 B5 sub5

Merge Two DataFrames on Multiple Keys

To merge two DataFrames on multiple keys, provide a list of column names to the on parameter.

Example

```
import pandas as pd

left = pd.DataFrame({'id': [1, 2, 3, 4, 5],

'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],

'subject_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']

})

# Creating the second DataFrame

right = pd.DataFrame({

'id':[1,2,3,4,5],

'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],

'subject_id':['sub2','sub4','sub3','sub6','sub5']})


# Merging on multiple keys 'id' and 'subject_id'
```

```
result = left.merge(right, on=['id', 'subject_id'])

print(result)
```

Its output is as follows −

	id	Name_x	subject_id	Name_y
0	4	A4	sub6	B4
1	5	A5	sub5	B5

Merge Using 'how' Argument

The how argument determines which keys to include in the resulting DataFrame. If a key combination does not appear in either the left or right DataFrame, the values in the joined table will be NaN.

Merge Methods and Their SQL Equivalents

The following table summarizes the how options and their SQL equivalents −

Merge Method	SQL Equivalent	Description
left	LEFT OUTER JOIN	Use keys from left object
right	RIGHT OUTER JOIN	Use keys from right object
outer	FULL OUTER JOIN	Union of keys from both inner
	INNER JOIN	Intersection of keys

Example: Left Join

```
import pandas as pd

left = pd.DataFrame({

'id': [1, 2, 3, 4, 5],

'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],

'subject_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']
```

```
})
right = pd.DataFrame({
'id': [1, 2, 3, 4, 5],
'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
'subject_id': ['sub2', 'sub4', 'sub3', 'sub6', 'sub5']
})

# Merging DataFrames using the left join method
print(left.merge(right, on='subject_id', how='left'))
```

Its output is as follows −

	id_x	Name_x	subject_id	id_y	Name_y
0	1	A1	sub1	NaN	NaN
1	2	A2	sub2	1.0	B1
2	3	A3	sub4	2.0	B2
3	4	A4	sub6	4.0	B4
4	5	A5	sub5	5.0	B5

Example: Right Join

This example performs the right join operation on two DataFrames using the merge() method by setting the how='right'.

```
import pandas as pd
left = pd.DataFrame({
'id': [1, 2, 3, 4, 5],
'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
'subject_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']
})

right = pd.DataFrame({
'id': [1, 2, 3, 4, 5],
'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
'subject_id': ['sub2', 'sub4', 'sub3', 'sub6', 'sub5']
})

# Merging DataFrames using the right join method
print(left.merge(right, on='subject_id', how='right'))
```

Output

	id_x	Name_x	subject_id	id_y	Name_y
0	2.0	A2	sub2	1	B1
1	3.0	A3	sub4	2	B2
2	NaN	NaN	sub3	3	B3
3	4.0	A4	sub6	4	B4
4	5.0	A5	sub5	5	B5

Outer Merge (how='outer')

- An outer merge (also known as a full outer join) returns a DataFrame containing all rows from both the left and right DataFrames.
- If a merge key exists in one DataFrame but not the other, the corresponding columns from the non-matching DataFrame will be filled with NaN (Not a Number) values in the result.
- It is analogous to a SQL FULL OUTER JOIN, effectively finding the union of the two DataFrames based on the specified key(s).

Example: Outer Join

```
import pandas as pd

left = pd.DataFrame({

'id': [1, 2, 3, 4, 5],
```

```
'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
'subject_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']
})

right = pd.DataFrame({
'id': [1, 2, 3, 4, 5],
'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
'subject_id': ['sub2', 'sub4', 'sub3', 'sub6', 'sub5']
})
# Merging the DataFrames using the outer join
print(left.merge(right, how='outer', on='subject_id'))
```

Output

	id_x	Name_x	subject_id	id_y	Name_y
0	1.0	A1	sub1	NaN	NaN
1	2.0	A2	sub2	1.0	B1
2	3.0	A3	sub4	2.0	B2
3	4.0	A4	sub6	4.0	B4
4	5.0	A5	sub5	5.0	B5

5 NaN NaN sub3 3.0 B3

Inner Join

Joining will be performed on index. Join operation honors the object on which it is called. So, a.join(b) is not equal to b.join(a).

Inner Merge (how='inner')

- An inner merge returns a DataFrame containing only the rows where the merge key(s) exist in both the left and right DataFrames.
- It is analogous to a SQL INNER JOIN, effectively finding the intersection of the two DataFrames based on the specified key(s).
- Rows that do not have a match in either DataFrame are excluded from the result.

Example

```
import pandas as pd

left = pd.DataFrame({

'id': [1, 2, 3, 4, 5],

'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],

'subject_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']

})

right = pd.DataFrame({
```

```
'id': [1, 2, 3, 4, 5],

'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],

'subject_id': ['sub2', 'sub4', 'sub3', 'sub6', 'sub5']

})


# Merge the DataFrames using the inner join method

print(left.merge(right, on='subject_id', how='inner'))
```

Output

	id_x	Name_x	subject_id	id_y	Name_y
0	2	A2	sub2	1	B1
1	3	A3	sub4	2	B2
2	4	A4	sub6	4	B4
3	5	A5	sub5	5	B5

The join() Method in Pandas

Pandas also provides a DataFrame.join() method, which is useful for merging DataFrames based on their index. It works similarly to DataFrame.merge() but is more efficient for index-based operations.

Syntax

DataFrame.join(other, on=None, how='left', lsuffix='', rsuffix='')

Example

```
import pandas as pd
left = pd.DataFrame({
'id': [1, 2, 3, 4, 5],
'Name': ['A1', 'A2', 'A3', 'A4', 'A5'],
'subject_id': ['sub1', 'sub2', 'sub4', 'sub6', 'sub5']
})
right = pd.DataFrame({
'id': [1, 2, 3, 4, 5],
'Name': ['B1', 'B2', 'B3', 'B4', 'B5'],
'subject_id': ['sub2', 'sub4', 'sub3', 'sub6', 'sub5']
```

```
})

# Merge the DataFrames using the join() method

result = left.join(right, lsuffix='_left', rsuffix='_right')

print(result)
```

Python Pandas - Pivoting

Pivoting in Python Pandas is a powerful data transformation technique that reshapes data for easier analysis and visualization. It changes the data representation from a "long" format to a "wide" format, making it simpler to perform aggregations and comparisons.

This technique is particularly useful when dealing with time series data or datasets with multiple columns. Pandas provides two primary methods for pivoting −

pivot(): Reshapes data according to specified column or index values.

pivot_table(): It is a more flexible method that allows you to create a spreadsheet-style pivot table as a DataFrame.

Pivoting with pivot()

The Pandas df.pivot() method is used to reshape data when there are **unique values for the specified index and column pairs**. It is straightforward and useful when your data is well-structured **without duplicate entries for the index/column** combination.

Example

```
import pandas as pd

df = pd.DataFrame({"Col1": range(12),"Col2": ["A", "A", "A", "B", "B","B", "C", "C", "C", "D", "D", "D"],

"date": pd.to_datetime(["2026-05-20", "2026-05-21", "2026-05-22"] * 4)})

print(df)

# Pivot the DataFrame

pivoted = df.pivot(

index="date",              # The column(s) to use as row labels
columns="Col2",            # The column(s) to use as column labels

values="Col1"              # The column to aggregate

)

print('Pivoted DataFrame:\n', pivoted)
```

Output

	Col1	Col2	date
0	0	A	2025-06-20
1	1	A	2025-06-21
2	2	A	2025-06-22
3	3	B	2025-06-20

4	4	B	2025-06-21
5	5	B	2025-06-22
6	6	C	2025-06-20
7	7	C	2025-06-21
8	8	C	2025-06-22
9	9	D	2025-06-20
10	10	D	2025-06-21
11	11	D	2025-06-22

Pivoted DataFrame:

Col2	A	B	C	D
date				
2025-06-20	0	3	6	9
2025-06-21	1	4	7	10
2025-06-22	2	5	8	11

Note: The pivot() method requires that the index and columns specified have unique values. If your data contains duplicates, you should use the pivot_table() method instead.

Pivoting with pivot_table()

The pivot() method is a straightforward way to reshape data, while pivot_table() offers flexibility for aggregation, making it suitable for more complex data manipulation tasks. This is particularly useful for summarizing data when dealing with duplicates and requires aggregation of data.

Pivoting with Aggregation

The Pandas pivot_table() method can be used to specify an aggregation function. By default it calculates the mean, but you can also use functions like sum, count, or even custom functions for applying aggregation to the pivoting.

```
import pandas as pd

data = {
    'Course': ['CS', 'Civil', 'CS', 'Civil', 'CS'],
    'Year': [1, 2, 1, 1, 2],
    'Student': [100, 150, 120, 90, 180]
}
df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

pivot_df = df.pivot_table(
    values='Student',  # The column to aggregate
    index='Course',  # The column(s) to use as row labels
    columns='Year', # The column(s) to use as column labels
```

```
    aggfunc='sum'    # The aggregation function (e.g., 'sum', 'mean', 'count')
)

print("\nPivot Table (Total Student by Course and Year):")
print(pivot_df)
```

Output

Original DataFrame:

	Course	Year	Student
0	CS	1	100
1	Civil	2	150
2	CS	1	120
3	Civil	1	90
4	CS	2	180

Pivot Table (Total Student by Course and Year):

Year	1	2
Course		
CS	220	180
Civil	90	150